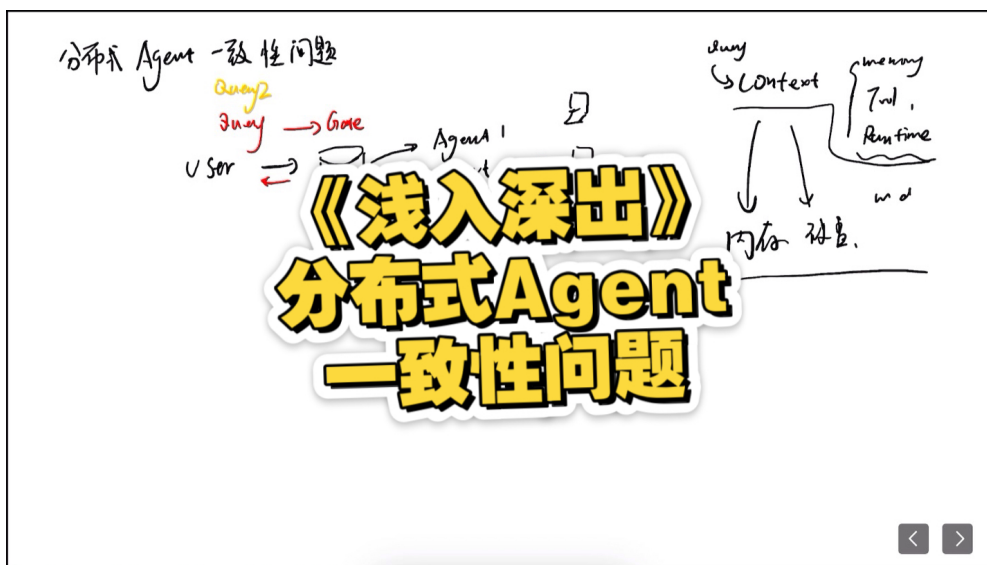


课程笔记

《浅入深出》Agent 系列之十五：分布式 Agent 一致性问题

SSS & Codex

2026 年 6 月 19 日



视频作者/频道：奇创喵

发布日期：2026 年 6 月 18 日

视频时长：12:12

视频链接：<https://www.bilibili.com/video/BV1B9jA6DEdU/>

目录

1 引言与上下文建模：分布式 Agent 为什么会遇到一致性问题	2
1.1 从单个 Agent 到分布式 Agent 服务	2
1.2 核心问题：同一用户的第二次 query 应该去哪里	2
1.3 上下文是什么：Memory、Tools、Tool Results 与 Runtime State	3
1.4 路由与上下文的耦合	4
1.5 本章小结	5
2 粘性路由：让 gateway 保存路由表	5
2.1 方案一：gateway 保存用户到 Agent 的映射	5
2.2 为什么这个方案看起来有效	7
2.3 代价一：gateway 路由表会膨胀	7
2.4 代价二：上下文绑定到某台机器	8
2.5 代价三：负载均衡逐渐失效	9
2.6 粘性路由的适用边界	10
2.7 本章小结	10
3 方案二：短状态 Agent、上下文落盘与按需恢复	10
3.1 从长期绑定转向短状态进程	10
3.2 上下文落盘：从易失内存到外部文件系统	11
3.3 本章小结	12
4 恢复流程：重新负载均衡并按用户 ID 加载记忆	12
4.1 用户第二天再来时发生什么	12
4.2 加载关键记忆，而非全量历史	13
4.3 按需摘取：让 Agent 再去找需要的旧内容	14
4.4 本章小结	14
5 MCP 化的记忆访问与最终架构	14
5.1 把 file system 记忆包装成 MCP tool	14
5.2 为什么 MCP 层重要	15
5.3 最终架构的平衡点	16
5.4 本章小结	16
6 总结与延伸	16
6.1 本讲的核心收束	16
6.2 概念边界：本讲没有展开正式容灾	17
6.3 面向实现的检查清单	17
6.4 本章小结	18

1 引言与上下文建模：分布式 Agent 为什么会遇到一致性问题

1.1 从单个 Agent 到分布式 Agent 服务

这一讲讨论的“一致性问题”，首先要放在一个具体的服务架构里理解：用户访问一个已经部署好的 Agent 服务时，请求先进入 gateway（网关），再由 gateway 路由到某一个 Agent 实例。这里的多个 Agent 实例可以运行同一份代码，内部能力和逻辑大体相同；差别在于它们是不同的进程，甚至可能分布在不同机器上。

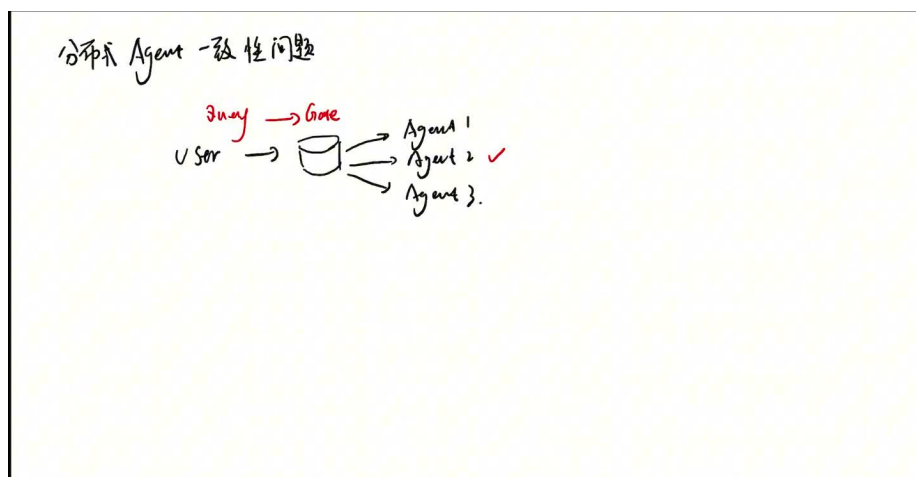


图 1: 分布式 Agent 的基本访问路径：用户请求先进入 gateway，再由 gateway 路由到某个 Agent 进程。¹

视频一开始强调，采用这种分布式结构的直接动机是吞吐量。若所有用户都压到单个 Agent 进程上，多用户并发时很容易出现阻塞；把请求分散到多个 Agent 实例后，系统可以同时处理更多用户请求。用一个直观类比说，单个 Agent 像只有一个服务窗口的柜台，多个 Agent 实例像开了多个窗口，gateway 则像入口处的分流人员。

知识补充

这里的 gateway 同时承担两类含义：一是网络入口，负责接收用户请求；二是路由决策点，决定某一次 query 交给哪个 Agent 实例处理。后面的“一致性问题”正是从这个路由决策点开始浮现的。

因此，分布式 Agent 的复杂性主要来自运行形态变化：同一个 Agent 服务被复制成多个实例，用户请求被动态分配到不同实例上。只要请求之间互不相关，这种架构很自然；一旦同一用户发起多轮连续 query，问题就变得敏感。

1.2 核心问题：同一用户的第二次 query 应该去哪里

视频把场景聚焦到一个用户身上。假设用户第一次访问 Agent 服务，发送第一个 query，记作 Q_1 。这个请求进入 gateway 后，gateway 随机或按某种负载均衡策略挑中 Agent2，于是 Q_1 由 Agent2 处理，并把 answer 返回给用户。

接着问题出现：当同一用户发起第二次 query，也就是 Q_2 时，gateway 应该把它路由到哪里？

¹ 视频画面时间区间：00:00:51–00:01:03。

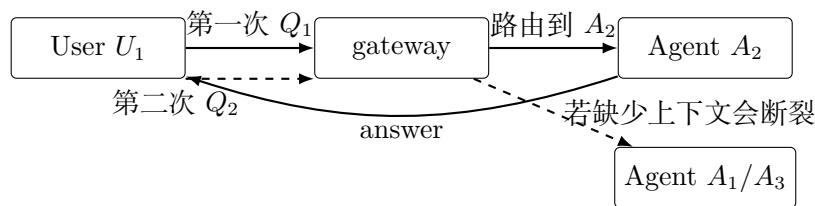


图 2: 第二次 query 的路由歧义: 若 Q_2 依赖 Q_1 的上下文, gateway 不能只看当前负载。

如果 Q_2 只是一个完全独立的新请求, 路由到 Agent1、Agent2 或 Agent3 似乎都可以。可多轮对话通常有连续性: 用户第二次提问可能依赖第一次的意图、约束、工具调用结果, 或者 Agent 在上一轮执行中形成的中间状态。此时, Q_2 的正确处理不只取决于当前文本, 还取决于上一轮留下来的上下文。

关键结论

本讲的“一致性问题”主要指会话上下文一致性: 同一用户的连续 query 需要看到同一份或等价恢复后的上下文, 否则 Agent 可能忘记前文、重复工具调用、误解用户意图, 或基于缺失状态生成不连续的回答。

这类一致性和数据库教材里的强一致性处在不同层面。这里关注一个更贴近 Agent 工程的问题: 用户多轮交互所依赖的状态, 怎样在多实例服务中被保留、传递或恢复。

易错点

不要把“分布式 Agent 一致性”直接等同于 Raft、Paxos 或数据库强一致性。视频这一段还没有展开共识协议, 也没有讨论正式的副本复制模型; 它讨论的是多实例 Agent 服务里, 用户上下文如何不丢失。

1.3 上下文是什么: Memory、Tools、Tool Results 与 Runtime State

为了理解为什么 Q_2 的路由会影响结果, 必须先明确 Agent 的 context (上下文) 包含什么。视频里提到, Agent 执行过程中有一个很重要的东西叫 context, 它包含对话记忆, 也可能包含 tools、tool results, 以及 Agent 自己运行时保留下来的状态。

可以把某个用户 u 的上下文粗略写成:

$$C_u = M_u + T_u + R_u + S_u$$

其中:

- C_u : 用户 u 的完整上下文, 是后续 query 能否连续理解的基础。
- M_u : memory, 即对话记忆和从历史交互中抽取出的用户相关信息。
- T_u : tool 相关记录, 包括工具使用过程和 tool results。
- R_u : runtime state, 即 Agent 运行时短期保留的状态。
- S_u : 摘要、索引或可恢复状态, 例如 memory markdown、主题索引、检索指针等。

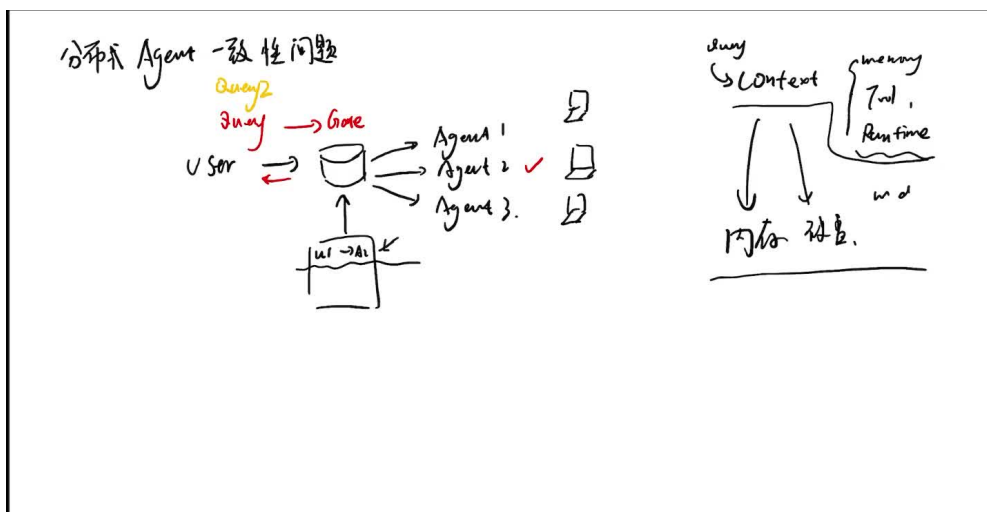


图 3: 连续会话需要保持上下文: 同一用户的后续 query 需要找到带有相关 context 的 Agent。²

Memory 比较容易理解: 用户上一轮问了什么、Agent 回答了什么、对话中已经确认了哪些偏好或约束, 都可能进入记忆。Tools 和 tool results 则来自 Agent 的工具调用链。例如上一轮 Agent 调用了某个检索工具、文件工具或外部服务, 返回的结果可能是下一轮推理的证据。Runtime state 更短暂, 可能包括 Agent 在执行流程中暂存的中间信息、管理器状态或上下文引擎维护的结构。

知识补充

可以把 Agent context 想成“工作台”。Memory 像工作台上的便签, tool results 像刚查到的资料, runtime state 像当前还没收拾完的草稿和临时变量。用户继续追问时, Agent 需要看到这些东西, 才能接着前面的工作往下做。

在单机架构下, 这件事相对容易。视频里说, 之前讲 Agent 时基本按单机视角展开, 因此可以把上下文保存成 Markdown、文件, 或者存到数据库里。因为只有一个运行位置, 或者至少开发者默认上下文就在当前 Agent 所在环境中, 读取和更新路径比较直接。

分布式架构改变了这个默认前提。Agent1、Agent2、Agent3 是不同进程, 甚至位于不同机器上。假如 Q_1 由 Agent2 处理, 而 Q_2 被 gateway 分给 Agent1, Agent1 默认看不到 Agent2 内存中的 context, 也未必能访问 Agent2 本机硬盘里的临时文件。于是连续对话所需的信息就可能断掉。

易错点

把上下文只放在某个 Agent 实例的内存或本机磁盘里, 会隐含一个强绑定假设: 后续 query 还会回到同一个实例。这个假设在单机或低并发测试中经常成立, 在真实分布式服务里很快会变成限制。

1.4 路由与上下文的耦合

到这里, 问题可以更清楚地建模: gateway 的路由策略不再只是“哪台机器空闲”。它还必须考虑“哪台 Agent 实例掌握这个用户的上下文”。如果 gateway 把 Q_2 发到 Agent2, Agent2 可能还保留着 Q_1 之后的 context, 连续性较好; 如果发到其他实例, 就需要额外机制来恢复上下文。

²视频画面时间区间: 00:01:34-00:02:03。

这也是后续方案展开的起点：一种显而易见的做法是让 gateway 保存路由表，例如记录 $U_1 \rightarrow A_2$ ，后续同一用户的 query 继续送到 Agent2。这样做能暂时保持上下文连续，因为 Agent2 里的 context engine 或管理器可以把新的 query 融入已有上下文，再交给 LLM 继续处理。

不过，本章先只建立问题的底层逻辑：多实例带来吞吐量，多轮对话带来上下文依赖，gateway 路由把二者连接在一起。真正困难的地方在于，路由决策和上下文保存一旦绑定，系统就要同时面对负载均衡、状态容量、长期会话和跨机器恢复这些工程约束。

关键结论

本讲的核心建模可以压缩成一句话：分布式 Agent 服务的关键问题，是同一用户的连续 query 能不能访问到正确的上下文。这个上下文可能来自 memory、tool results、runtime state，也可能需要从外部存储中恢复。

1.5 本章小结

本章对应视频开头到 00:02:35 左右的内容，完成了三个基础建模步骤。

- 第一，分布式 Agent 架构通过 gateway 把用户请求分发到多个同构 Agent 实例，主要目的是提升吞吐量并减少单实例阻塞。
- 第二，同一用户的第二次 query 会暴露路由问题：如果它依赖上一轮对话，随意分配到另一个 Agent 实例可能导致上下文断裂。
- 第三，Agent context 至少包括 memory、tools、tool results 和 runtime state。单机保存相对直接，分布式环境中则必须考虑这些状态如何跨进程、跨机器保持或恢复。

后续章节会沿着这个模型继续分析：先看 gateway 维护路由表的粘性路由方案，再讨论它在状态膨胀、机器绑定和负载均衡方面的代价，最后引出“短状态 Agent + 外部上下文落盘 + 按需恢复”的更稳妥架构。

2 粘性路由：让 gateway 保存路由表

2.1 方案一：gateway 保存用户到 Agent 的映射

上一章提出的问题是：同一个用户的第一次 query 已经由某个 Agent 实例处理，第二次 query 到来时，gateway 应该把它送到哪里？视频中给出的第一个直观方案是让 gateway 保存一张路由表。路由表记录“用户标识 $\rightarrow$ Agent 实例”的映射，例如：

$$\text{RouteTable}[U_1] = A_2$$

其中：

- U_1 表示示例用户 User1；
- A_2 表示第二个 Agent 实例；
- RouteTable 表示由 gateway 维护的路由表；
- 这个映射的工程含义是：只要 U_1 的会话仍然有效，后续请求继续送到 A_2 。

这样做的动机很直接。第一次 query 进入 gateway 时，gateway 可以随机选择、轮询选择，或者按照某种负载均衡策略选择一个 Agent 实例。假设它选择了 A_2 ，那么 A_2 在处理过程中会保留该用户的上下文：对话记忆、工具调用结果、运行时状态、临时中间结果等。第二次 query 到来时，gateway 查询路由表，发现 U_1 上一次被导流到了 A_2 ，于是继续把请求送到 A_2 。

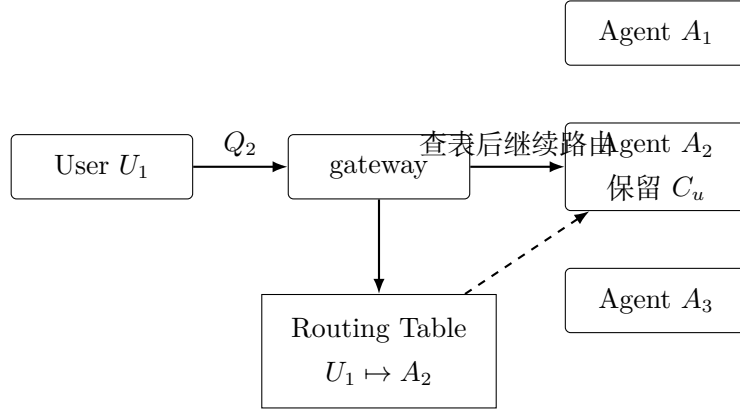


图 4: 粘性路由把用户到 Agent 实例的历史映射保存到 gateway 路由表中。

关键结论

粘性路由的核心思想是：让连续会话尽量落在同一个 Agent 实例上。这样第二次 query 可以直接复用该实例里仍然存在的上下文，避免把一个需要“接着聊”的请求发送到一个没有历史状态的实例。

在 A_2 内部，新的 query 会被放回 Agent runtime，由 context engine 或自定义 context manager 合并进现有上下文，再交给 LLM 继续处理。这里的关键点在于“合并”：第二次 query 不是孤立文本，它要和之前的 memory、tool result、runtime state 一起构成新的输入背景。用一个简化公式表示：

$$C_u^{(t+1)} = \text{merge}(C_u^{(t)}, Q_{t+1})$$

其中：

- $C_u^{(t)}$ 表示用户 u 在第 t 轮之后的上下文；
- Q_{t+1} 表示下一轮用户 query；
- $\text{merge}(\cdot)$ 表示把新请求并入已有上下文的管理逻辑；
- $C_u^{(t+1)}$ 表示更新后的上下文。

知识补充

这个方案很像传统 Web 系统里的 sticky session：负载均衡器记住某个用户曾经落到哪台应用服务器，后续请求继续发到同一台服务器。区别在于，Agent 的“会话状态”通常更重，因为它可能包含多轮对话记忆、工具调用历史、文件引用、执行环境状态和可恢复的中间推理材料。

2.2 为什么这个方案看起来有效

从单个用户的视角看，gateway 路由表确实解决了一个最明显的问题：第二次 query 不会被送到陌生的 Agent 实例。只要 A_2 的上下文仍然存在，用户就会感觉系统“记得刚才发生了什么”。例如，用户第一轮让 Agent 查资料、调用工具、生成初稿，第二轮说“把刚才第三点改成更正式的表达”，这个“刚才第三点”依赖前一轮上下文。粘性路由把第二轮请求送回 A_2 ，于是 A_2 有机会直接读取本地上下文完成续写。

该方案的吸引力来自三个工程优势：

- 实现路径短：gateway 增加一张路由表，Agent 继续使用原有本地上下文管理方式；
- 请求链路清晰：用户请求先到 gateway，gateway 查表后转发到指定 Agent；
- 短时间内体验好：连续 query 之间的时间间隔较短时，本地上下文大概率仍在内存或本机磁盘中。

关键结论

在短会话、低并发、用户停留时间可控的场景中，粘性路由是一个合理的第一步。它把“连续对话的一致性”转化成“同一用户继续访问同一 Agent 实例”的路由问题。

2.3 代价一：gateway 路由表会膨胀

问题从规模开始出现。gateway 保存路由表意味着每个活跃用户都可能占用一条映射记录。用户越多，路由表越大；会话保留时间越长，旧映射越难释放。视频明确指出，路由表不能无限膨胀。这个限制很现实：路由表本身需要内存、更新、过期、查询和一致性维护。它从一个“简单映射”变成了一个必须被管理的状态系统。

设系统有 N 个曾经访问过的用户，平均每条路由记录需要 b 字节，则 gateway 至少要承担：

$$M_{\text{route}} = N \times b$$

其中：

- M_{route} 表示路由表占用的内存或缓存空间；
- N 表示仍然保留路由记录的用户数量；
- b 表示单条记录的平均存储成本，包括用户 ID、Agent ID、时间戳、过期策略元数据等。

这条公式很粗糙，却足以说明风险：当 N 增长时，路由表成本线性增长。如果系统把“曾经来过的用户”都长期留在路由表中，gateway 就会从轻量入口变成状态堆积点。

易错点

路由表不是免费的缓存。它越大，越需要过期策略、容量控制、故障恢复和跨 gateway 同步。若把它当成永久用户状态库，gateway 会承担它原本不该承担的长期记忆职责。

2.4 代价二：上下文绑定到某台机器

更麻烦的是，路由表记录的是“用户应该回到哪个 Agent 实例”，而该 Agent 实例常常运行在某台具体机器上。视频中特别提到：Agent2 可能运行在一台服务器上，其他 Agent 运行在另外的机器上。如果 U_1 的 context 保存在 A_2 的内存里，或者保存在 A_2 所在机器的本地硬盘里，那么这个用户就被机器绑定了。

机器绑定带来的后果是：用户连续性依赖某个具体实例的存活和可达性。只要 U_1 后续还想复用历史上下文，gateway 就倾向于继续把它送回 A_2 。这在短期内很方便；随着时间变长，它会把一部分用户长期钉在特定机器上。

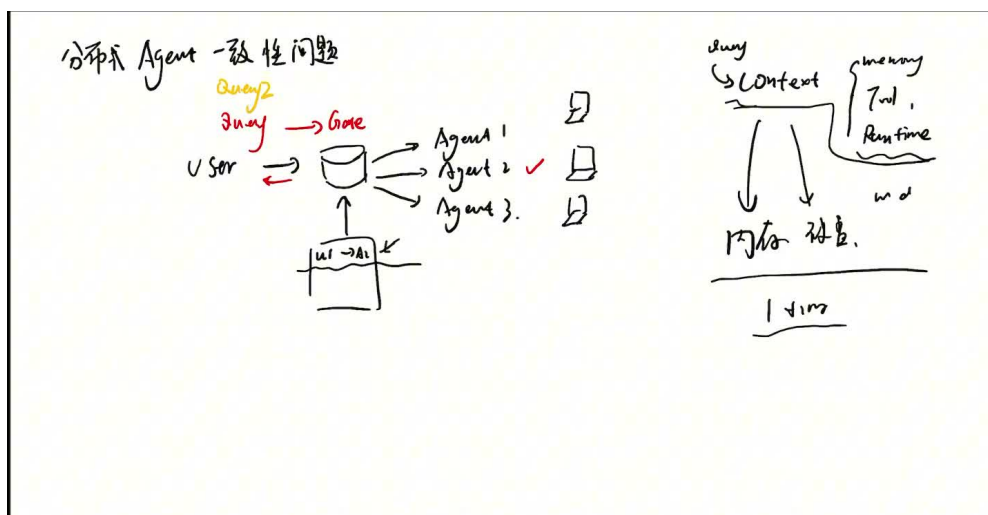


图 5：粘性路由的代价：路由表和上下文不能无限保留，长时间无操作后需要释放内存与路由占用。

3

知识补充

可以把 gateway 想象成酒店前台，把 Agent 实例想象成房间。前台记住“这位客人住 202 房间”很合理，因为客人的行李还在 202。但客户档案、历史订单、会员权益若也只放在 202 房间，就会让换房、维修、扩容都变得困难。Agent 的长期上下文更适合进入外部存储，本地实例适合保留短期工作台状态。

这里还藏着一个常见误解：把 context 放到“本机磁盘”并不能解决分布式问题。内存和本机磁盘都属于具体机器的局部资源。只要其他 Agent 实例不能稳定访问这份上下文，系统仍然要求用户回到原机器。

易错点

“已经落到硬盘”不等于“已经可被分布式系统共享”。本机硬盘只是把状态从易失内存移到了同一台机器的持久介质；它仍然没有变成所有 Agent 都能按用户 ID 读取的共享上下文。

³视频画面时间区间：00:03:17-00:06:42。

2.5 代价三：负载均衡逐渐失效

粘性路由最致命的代价，是它会削弱负载均衡。负载均衡的目标是把请求按当前资源状况分散到多个 Agent 实例上，使 CPU、内存、队列长度和外部工具调用压力保持相对均匀。粘性路由加入后，gateway 的选择空间变小了：对于已有路由记录的用户，它优先服从历史绑定，而非当前负载。

视频给了一个非常直观的规模例子：假设系统有 100 万用户，使用 1 万个 Agent 实例承载这些用户。第一次访问时，gateway 可以把 100 万用户平均导流到 1 万个 Agent 上，每个 Agent 大约承载：

$$\frac{1,000,000}{10,000} = 100$$

其中：

- 1,000,000 表示 100 万用户；
- 10,000 表示 1 万个 Agent 实例；
- 100 表示初次均匀分配时，每个 Agent 平均绑定约 100 个用户。

到这里看起来很均衡。问题发生在后续访问阶段：假如过了一段时间，只有一部分用户继续活跃，另一部分用户很少再访问。活跃用户未必均匀分布在所有 1 万个 Agent 上。视频举的情况是，可能只有 5000 个 Agent 还在努力工作，剩下 5000 个 Agent 基本闲置。此时系统虽然“名义上”有 1 万个 Agent，实际可用负载被压在其中一半实例上。

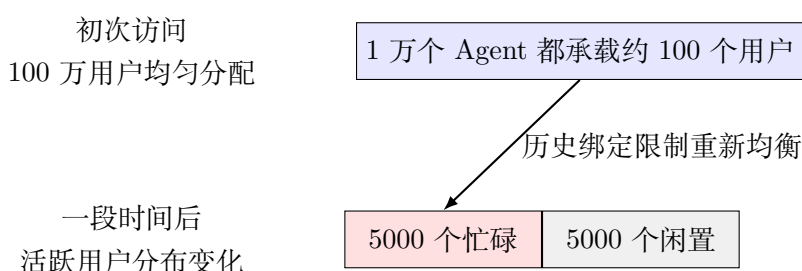


图 6：粘性路由会把初始分配变成后续约束；当活跃用户集合变化时，真实负载可能集中在部分 Agent 上。

关键结论

粘性路由会把“历史分配”变成“未来约束”。用户第一次被分到哪里，后续就倾向于继续回到哪里；当活跃用户集合变化时，系统很难重新把真实负载摊平。

这个例子的关键在于分布变化；数字只是把这种变化具体化。第一次访问时的均匀分配只保证了初始状态；它无法保证未来仍然均匀。用户活跃度具有时间性：有人只来一次，有人持续高频访问，有人隔天回来，有人在某个高峰期突然变活跃。若所有这些行为都被旧路由记录锁住，gateway 就失去了根据当前负载重新选择 Agent 的能力。

易错点

负载均衡失效通常不会表现为所有机器同时变慢。更常见的情况是：一部分 Agent 排队、内存上涨、响应变慢，另一部分 Agent 空闲。用户看到的是系统抖动，运维看到的是资源利用率割裂。

2.6 粘性路由的适用边界

粘性路由适合解决短期连续会话问题。比如用户正在一轮密集对话中，前后 query 间隔只有几秒到几分钟，Agent runtime 里还有刚刚产生的临时状态，这时把请求送回原 Agent 很自然。它提供的是“短期会话连续性”，适合做热路径优化。

它的边界也同样清楚：

- 用户规模扩大后，gateway 路由表会变成必须治理的状态；
- 上下文留在 Agent 内存或本机磁盘时，用户会被绑定到具体机器；
- 用户活跃度变化后，历史绑定会压缩负载均衡的选择空间；
- 一旦某个 Agent 实例、机器或本地磁盘出现问题，相关用户的连续体验会受到影响。

因此，粘性路由更像一个过渡层：它能让短时间内的 query 回到原 Agent，却无法承担长期记忆、全局负载调度和跨机器恢复的全部责任。后续章节要引入的方案，正是把“短期路由保持”和“长期上下文持久化”拆开处理。

知识补充

TTL (Time To Live, 生存时间) 在这里承担资源管理策略的角色。路由记录保留多久，决定了系统更偏向连续体验，还是更偏向资源释放和重新均衡。保留过短会打断连续会话；保留过长会放大路由表和机器绑定问题。

2.7 本章小结

本章讨论了视频中的第一个方案：gateway 保存路由表，把用户 U_1 后续 query 继续导流到上一次处理它的 Agent 实例 A_2 。这个方案能让 Agent 复用仍在本地的 context，由 context engine 或 manager 把新 query 合并进旧上下文，再交给 LLM 继续处理。

它的问题也很硬：gateway 路由表不能无限膨胀，Agent 内存或本机磁盘里的 context 不能无限增长，长期把用户绑定到某台机器会让负载均衡逐渐失效。100 万用户、1 万 Agent 的例子说明了这一点：初次访问时可以平均分配；后续只有部分用户活跃时，可能只有 5000 个 Agent 忙碌，另外 5000 个 Agent 闲置。下一章需要解决的核心问题，就是怎样释放长期路由绑定，同时仍然让用户历史上下文可恢复。

3 方案二：短状态 Agent、上下文落盘与按需恢复

3.1 从长期绑定转向短状态进程

前一章已经说明，单纯依赖 gateway 路由表会把用户长期绑定到某一个 Agent 实例上。这个做法能维持短时间内的对话连续性，却会逐步牺牲负载均衡能力：用户越多、会话越长、上下文越大，路由表和 Agent 本地状态就越难维护。

视频给出的工业折中方案是“结合式”的：用户刚来时，gateway 仍然可以建立短期路由，例如记录 $U_1 \rightarrow A_2$ ；用户持续交互时，请求继续送到 A_2 ，这样能避免每一轮都重新恢复上下文。等用户一段时间没有操作，例如关闭客户端，或者半小时、一小时内没有新的指令，系统就释放这条绑定关系。

这个等待时间可以记作一个超时阈值 τ 。如果用户 u 的最后一次访问时间为 $t_{\text{last}}(u)$ ，当前时间为 t ，当

$$t - t_{\text{last}}(u) > \tau$$

系统就可以把该用户视为暂时离线，并执行两件事：

- 从 gateway 的 routing table 中移除该用户到某个 Agent 实例的绑定。
- 把该用户的上下文从 Agent 运行时环境中持久化到外部存储。

其中 τ 表示空闲超时阈值； $t_{\text{last}}(u)$ 表示用户 u 最近一次发起 query 的时间； t 表示当前系统时间。这个公式表达的核心很朴素：用户长时间没有新动作时，占住路由表和 Agent 内存的收益会下降，释放资源的收益会上升。

知识补充

可以把它类比为酒店前台分房。客人刚入住时，前台需要知道他住在哪个房间；客人退房或长时间离店后，房间应该释放给其他客人。用户资料、账单和偏好应保存在中央系统里，而不应该永远压在某一个房间里。

这里的“短状态 Agent”指 Agent 实例可以在活跃会话期间保留必要状态，但不会永久保存某个用户的全部信息。它介于完全无状态和长期有状态之间：短期为了效率保留上下文，长期为了可扩展性把上下文交给外部存储。

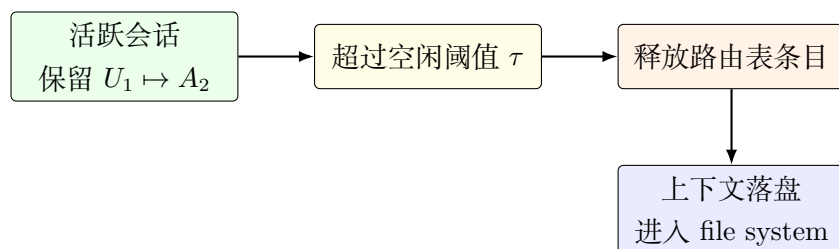


图 7: 短状态生命周期：活跃时保留路由，空闲超时后释放绑定并持久化上下文。

关键结论

方案二的核心是把“请求应该路由到哪里”和“用户上下文保存在哪里”拆开。gateway 可以继续做负载均衡，Agent 实例只承担短期运行状态，长期记忆和历史内容落到独立的 file system 或存储服务里。

3.2 上下文落盘：从易失内存到外部文件系统

视频中说“context 落盘”或“content 落盘”，含义是把原本停留在 Agent 内存、运行时上下文或本机临时存储中的内容，写入更持久的外部存储。这里的“落盘”不一定只指物理硬盘，它更准确地表示：从易失运行态迁移到可在未来重新读取的存储层。

如果上下文很小，确实可以考虑放进数据库字段中。但视频提醒，一个数据库字段或单个值通常有存储上限；当用户经历很多轮对话，memory 可能变得非常大，把完整历史压成一个巨大的二进制串再塞进数据库，并不自然。更常见的做法是把上下文整理成静态文件，放到独立于 Agent 实例之外的 file system 服务中。

这个静态文件可以是 Markdown 摘要、结构化索引、压缩后的文本文件，也可以是其他序列化格式。视频里的 ASR 对具体格式有噪声，因此这里不指定某个特定工具。真正重要的是架构边界：文件服务独立于任意一个 Agent 进程，Agent 实例重启、迁移或释放本地状态后，用户上下文仍能被后续实例读取。

易错点

不要把“上下文落盘”理解成“把所有历史原封不动塞回下一轮 prompt”。持久化解决的是长期保存和可恢复性；每次推理时加载多少内容，还要受 token budget、相关性、隐私边界和延迟约束影响。

在这个方案里，用户上下文 C_u 可以被分层保存：

- 关键记忆：用户偏好、长期目标、已确认事实、重要约束。
- 主题索引：历史对话围绕哪些任务展开，哪些段落和文件可能相关。
- 完整历史：更长的 transcript、工具结果、细节记录，供后续按需检索。
- 临时运行态：只在活跃窗口内有意义，超时后可以丢弃或摘要化。

这种分层让系统既保留连续性，又避免每次恢复都拖入全部历史。直观地说，memory markdown 像目录和摘要，完整 transcript 像档案库。用户重新回来时，Agent 先看目录和摘要；只有问题需要时，再去档案库翻具体页。

3.3 本章小结

方案二把 Agent 实例设计成无状态或短状态进程。活跃会话期间，gateway 可以暂时维持用户到某个 Agent 实例的路由；用户空闲超过阈值后，系统释放路由表条目，把上下文落盘到独立 file system。这个设计用短期粘性路由换取响应效率，用长期外部存储换取可扩展性和可恢复性。

4 恢复流程：重新负载均衡并按用户 ID 加载记忆

4.1 用户第二天再来时发生什么

视频接着讨论用户第二天再次访问的场景。此时，gateway 的路由表里已经没有这个用户的记录，因为上一轮会话超时后绑定关系被释放。新的 query 到达 gateway 后，gateway 按照正常负载均衡机制选择一个合适的 Agent 实例。

这一次，用户 U_1 可能被导流到 A_1 ，而上一轮处理它的实例可能是 A_2 。从负载均衡角度看，这是好事：系统重新拥有自由分配请求的能力。从上下文连续性角度看， A_1 需要恢复 U_1 的历史记忆，否则它面对的只是一个孤立 query。

恢复流程可以拆成四步：

⁴视频画面时间区间：00:08:21–00:09:28。

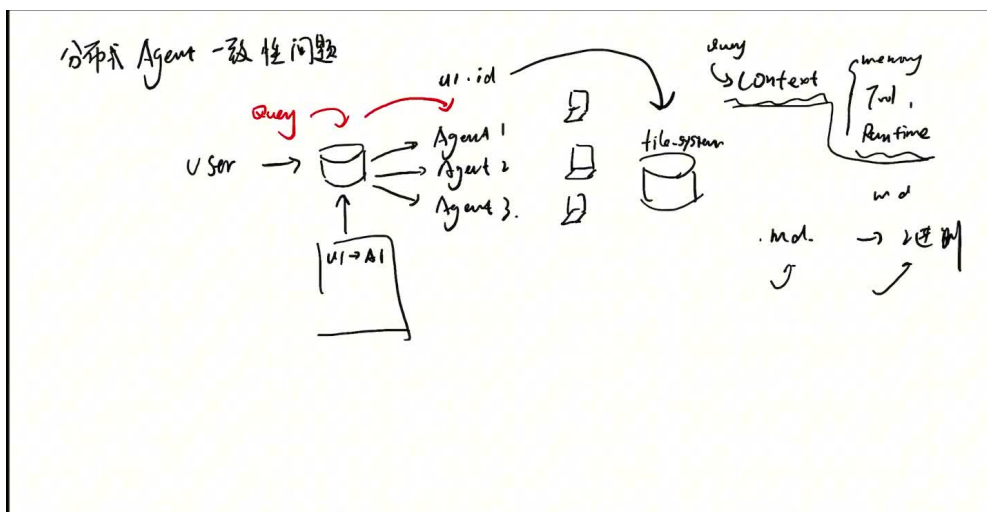


图 8: 路由过期后的恢复路径: 新 Agent 通过用户 id 从文件系统取回关键上下文, 再加载到当前运行时。⁴

1. 用户发送新的 query, 请求进入 gateway。
2. gateway 查 routing table, 发现没有该用户的现存绑定。
3. gateway 按负载均衡策略把请求分配给当前合适的 Agent 实例。
4. 新 Agent 实例根据 user id 或 uuid 到 file system 中查找该用户的历史上下文文件。

这里的关键主键是用户身份标识, 例如 user id 或 uuid。Agent 不再依赖“上一次是哪台机器处理的”来寻找记忆, 而是依赖“这是谁的记忆”。这使上下文恢复从机器绑定转为用户绑定。

关键结论

恢复流程的中心动作是按用户 ID 加载记忆。只要外部存储能根据用户 ID 找到对应上下文, 新的 Agent 实例就可以接手旧会话, 而 gateway 仍然可以按照负载均衡选择实例。

4.2 加载关键记忆, 而非全量历史

视频明确提醒, 完整历史对话通常不会全部加载进 context。原因很直接: 完整历史可能很长, 包含大量当前 query 不需要的内容; 全部加载会浪费上下文窗口, 提高延迟, 也可能把噪声带入推理。

因此, 新 Agent 实例恢复时更合理的动作是 first load: 先加载关键上下文, 例如 memory mark-down、重要信息摘要、主题索引, 以及能指向完整历史的检索线索。之后, Agent 根据当前 query 判断是否需要查看更多旧内容。

知识补充

first load 可以理解成“先打开用户档案的首页”。首页上有用户偏好、关键事实、最近主题、历史材料索引。Agent 先用这些信息恢复对用户的基本理解; 如果当前问题需要细节, 再顺着索引查更长的历史记录。

这种方式也更符合 LLM 的上下文经济性。LLM 的输入窗口有限，设窗口容量为 B ，一次恢复加载的内容大小为 L ，当前 query 与系统提示、工具说明等其他必要内容大小为 Q 。实际可用空间需要满足：

$$L + Q \leq B$$

其中 B 表示本次推理可用的上下文窗口预算； L 表示恢复时加载的历史记忆大小； Q 表示当前请求、系统提示、工具描述等其他必要输入。若 L 被完整历史占满，当前 query 的有效推理空间反而会被挤压。

易错点

恢复上下文时，最危险的误解是“越多越好”。对 Agent 来说，恢复质量取决于相关性和结构化程度。加载一份清晰的 memory markdown，常常比塞入一大段未经整理的历史 transcript 更稳定。

4.3 按需摘取：让 Agent 再去找需要的旧内容

first load 之后，如果 Agent 判断关键记忆不够，它可以发起“加载旧记忆”的动作。视频把这一步和此前讲过的 Agent memory 管理连接起来：当 Agent 收到 query 时，它会先检查记忆里有没有相关内容；如果有，再把相关内容提取出来。

在外部化上下文架构中，这个过程变成按需检索。完整 transcript 或长历史可以继续保留在 file system 中，Agent 只在需要时读取相关片段。这样做的收益有三点：

- 恢复速度更可控：初次加载只取关键摘要，避免每次都读大文件。
- 推理输入更干净：只把与当前 query 相关的旧内容带入 LLM。
- 存储边界更清楚：完整历史归 file system 管，运行时 context 只保留当前推理所需内容。

这也是“短状态 Agent”能成立的原因。Agent 实例不需要永久携带用户全量历史；它只需要在处理当前 query 时，拿到足够的关键记忆和可检索入口。

4.4 本章小结

恢复流程的重点不在于把用户塞回旧机器；更合理的动作是让 gateway 重新执行负载均衡，再由新 Agent 按用户 ID 去 file system 找回记忆。恢复时优先加载 memory markdown 和主题索引，完整历史按需摘取。这样，用户体验上仍然像连续对话，系统内部则重新获得实例调度的自由度。

5 MCP 化的记忆访问与最终架构

5.1 把 file system 记忆包装成 MCP tool

视频进一步提出，如果记忆存在 file system 中，就可以把这套访问能力包装成一个 MCP tool。这里的 MCP 可以理解为一种工具接口层：LLM 或 Agent 不直接关心底层文件路径、存储格式和检索细节，而是通过标准化工具去查询、读取和补充历史记忆。

换句话说，file system 负责保存内容，MCP tool 负责把“按用户 ID 找文件”“按主题查索引”“读取相关 transcript 片段”这些能力暴露给 Agent。Agent 处理 query 时，可以先通过 first load 拿到关键上下文；当上下文不够时，再通过 MCP tool 读取更多历史。

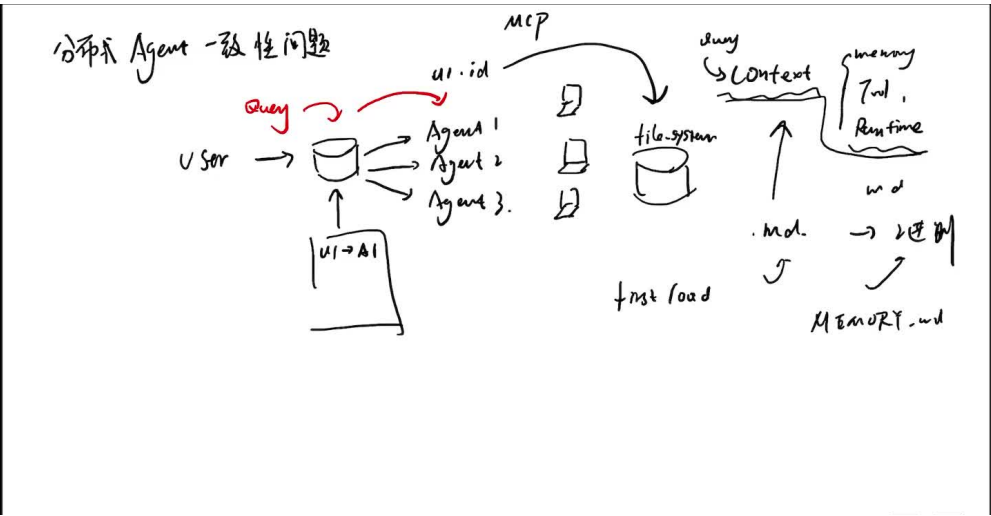


图 9: 按需记忆加载机制：首次加载关键上下文，后续通过 MCP 从文件系统按需取回更完整的记忆。⁵

关键结论

最终架构可以概括为：gateway 负责导流，Agent 实例保持无状态或短状态，file system 保存长期上下文，MCP tool 向 Agent/LLM 暴露可控的记忆检索能力。

这种设计让每层职责更清楚：

- gateway：处理入口流量和负载均衡。
- Agent 实例：处理当前 query、维护短期运行态、调用 LLM 和工具。
- file system：持久保存用户历史、memory markdown、transcript、工具结果等内容。
- MCP tool：把外部记忆转化为 Agent 可调用的查询和读取能力。

5.2 为什么 MCP 层重要

如果没有 MCP 层，Agent 可能需要在业务代码里硬编码文件路径、存储格式、检索逻辑和权限判断。随着记忆类型变多，Agent 代码会越来越像一个存储系统客户端，职责变得混乱。

MCP 层的价值在于把记忆访问抽象成可治理的工具能力。Agent 可以表达“查找这个用户与当前 query 相关的旧记忆”，工具层再负责实际检索。工程实现中，这一层还可以承载读取范围控制、日志记录和格式转换等治理能力；这些属于实现扩展。

知识补充

MCP tool 像一个档案管理员。Agent 不需要冲进档案室翻所有柜子，只需要说明要找哪类材料；档案管理员根据用户 ID、主题索引和当前问题，取出合适的片段交给 Agent。

⁵ 视频画面时间区间：00:09:45-00:10:49。

这也解释了视频中“剩余 transcript 部分保留在 file system 中，以 MCP 的形式提供给 LLM”的含义。LLM 不必在 first load 时吞下所有 transcript；它可以在推理过程中通过工具访问需要的片段。这样，外部记忆既保持持久，又不会压垮单轮上下文。

5.3 最终架构的平衡点

视频收束时说，这套架构支持了分布式访问功能：用户先经过 gateway 导流到某个 Agent 实例；每个 Agent 实例基本属于无状态或短状态进程，不会一直保存用户信息；它会从 file system 中拿到对应内容，并通过 MCP 获得后续记忆和补充。

这套架构实际是在三个目标之间做平衡：

- 吞吐量：gateway 可以继续把用户分配给当前更合适的 Agent 实例。
- 持久性：用户记忆离开单个 Agent 进程，进入外部 file system。
- 连续性：Agent 通过 first load 和 MCP 检索恢复足够上下文。

它解决的是会话上下文连续性问题。用户不会因为第二天被路由到另一个 Agent 实例，就完全丢失历史；系统也不会因为要保留历史，就把所有用户永远绑在固定机器上。

易错点

这里的“无状态 Agent”要谨慎理解。用户状态并没有消失，只是从 Agent 进程内部转移到了外部存储和 MCP 访问层。系统整体仍然有状态，只是状态的归属从单个运行实例变成了可共享、可恢复的外部组件。

5.4 本章小结

MCP 化的记忆访问把 file system 中的历史上下文变成 Agent 可调用、可治理、可按需读取的工具能力。最终架构中，gateway 做负载均衡，Agent 实例保持短状态，file system 负责持久化，MCP tool 负责记忆检索。这个分层让分布式 Agent 同时获得更好的吞吐量、用户数据持久性和会话连续性。

6 总结与延伸

6.1 本讲的核心收束

本讲围绕一个非常具体的问题展开：同一用户的多轮 query 在分布式 Agent 服务中如何保持上下文连续。视频先展示了粘性路由的直观方案，再指出长期绑定会带来路由表膨胀、Agent 本地状态增长和负载均衡失效。随后给出的更合理架构，是短状态 Agent 加外部化上下文。

可以把整套方案压缩成一句话：短期用路由表保持活跃会话连续，长期把用户上下文落盘到外部 file system，并通过 user id 和 MCP tool 按需恢复。

这个方案的工程含义很明确。Agent 服务真正需要长期保存的，不应该绑死在某个进程或某台机器上；Agent 实例只负责当前执行，长期记忆由外部存储接管。用户再次回来时，系统先重新负载均衡，再按用户身份恢复记忆。

⁶视频画面时间区间：00:10:56–00:11:36。

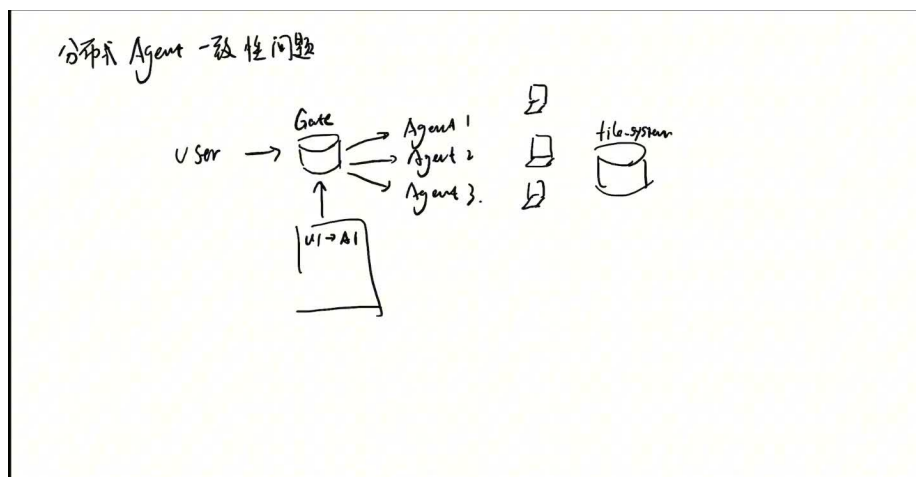


图 10: 最终总览: gateway 负责导流, Agent 负责执行, 外部文件系统保存可迁移的上下文与记忆。
6

6.2 概念边界: 本讲没有展开正式容灾

视频最后给出一个重要 caveat: Agent 一致性问题很多时候需要结合容灾处理一起讲。因为在真实系统中, 还会出现分区出错、节点故障、机器损坏等情况。到了那一层, 问题会从“用户第二天被分到新 Agent 时怎么恢复记忆”, 进一步变成“某些组件故障时, 数据如何仍然保持一致和可恢复”。

易错点

本讲没有提供正式共识协议、复制日志、故障切换等容灾细节。把这些内容强行补进本章, 会超出视频范围。视频只把容灾作为更深一致性讨论的伏笔, 并说明下一期会详细介绍。

因此, 本章的结论应该被限定在“会话上下文一致性”这一层。它讲清楚了 Agent 进程、gateway、file system、MCP tool 之间如何分工; 它还没有回答在 file system 故障、网络分区、多个副本写入冲突时如何保持数据一致。这些问题属于更深一层的灾难恢复和故障场景设计。

6.3 面向实现的检查清单

如果要把视频里的方案落到工程实现中, 可以把关键检查点列成以下几项:

- gateway 是否支持短期 sticky routing, 并能在用户空闲后释放 routing table 条目。
- Agent 实例是否避免长期保存用户全量历史, 只保留活跃会话所需的短期状态。
- 用户上下文是否按 user id 或 uuid 写入外部 file system。
- 持久化内容是否区分 key memory、topic index、完整 transcript 和工具结果。
- first load 是否只加载关键上下文, 而非全量历史。
- MCP tool 是否提供按需检索旧记忆的能力, 并能控制读取范围。
- 后续容灾设计是否单独处理节点故障、分区异常和存储不可用场景。

这份清单也揭示了一个容易被忽略的点：分布式 Agent 的一致性无法靠单个开关解决，它是一组边界设计。路由边界、存储边界、记忆检索边界、故障恢复边界都需要明确。边界越模糊，系统越容易在规模扩大后出现“某个用户只能由某台机器处理”的隐性耦合。

6.4 本章小结

本讲的最终结论是：分布式 Agent 要同时追求吞吐量和连续性，就需要把用户上下文从单个 Agent 实例中外部化。短状态 Agent 让实例可以被自由调度；上下文落盘让用户记忆可以长期保存；按 user id 恢复和 MCP tool 检索让新实例能接续旧会话。更深层的一致性问题还要进入容灾和故障场景，例如分区出错或机器故障时如何保持数据一致，这部分需要在后续专题中继续展开。